提交材料 • 作战方案

四件套万全之策

技术方案 · 技术报告 · 佐证材料 · 商业报告书

项目：**智维 Agent（SmartOpsAgent）** 面向 Linux 运维场景的可信智能体

核心判断

四份材料不是四篇文章，
而是同一个论证被四把不同的尺子量。

赛事	**2026 第八届全球校园人工智能算法精英大赛 • 河南赛区 • 算法创新赛**
文档性质	**写作作战方案（不是材料本身）**
版本	**v1.0 • 2026-11-02**
阅读用时	**全文约 20 分钟；赶时间只读第 0、3、6、8 节**

打印建议：A4 双面（长边翻转）· 缩放 100% · 黑白即可，全册不使用深色底，省墨且不糊

怎么用这份方案

适用：2026 第八届全球校园人工智能算法精英大赛 · 河南赛区算法创新赛
项目：智维 Agent（SmartOpsAgent）—— 面向 Linux 运维场景的可信智能体
版本：v1.0　|　本文是"怎么写"的作战方案，不是材料本身

目录

节	内容	你会拿到什么
0	**一句话战略**	全局判断，以及三种“死法”的对应解法
1	**评委怎么读这四份材料**	四把尺子：谁在读、找什么、一票否决点
2	**总架构：一源•四层•三线**	从事实源到四份材料的派生关系
3	**单一事实源（口径台账）**	9 条可用底数 + 5 项待测 + 禁用口径
4	**分工矩阵：一份只答一个问题**	四份材料零重复的划界表
5	**逐份作战方案**	四份材料的目录大纲、页数预算、扣分点
6	**三条红线与标准话术**	数据集 / 实验 / 规则基线的统一说法
7	**佐证材料清单**	L1 / L2 / L3 三级证据打勾表
8	**一致性矩阵**	同一事实在四份材料里的四种写法
9	**七天作战排期**	含依赖关系的写作顺序
10	**交付前 30 项自检**	打印前逐条过一遍
11	**需要你确认的 4 件事**	决定写法的决策点
附	**与既有材料的关系**	答辩稿体系如何复用

三句话读完

一、先做「口径台账」（第 3 节），再写任何一份材料——否则四份必然互相打脸。

二、四份材料各答一个问题：为什么这么设计 / 实际做成什么 / 凭什么信 / 谁付钱，**内容不许重复。**

三、本项目有三处必须提前备好话术：数据集仅 14 条且全 DRAFT、实验 NOT RUN、多个组件是规则基线（第 6 节）。

0. 一句话战略

四份材料不是四篇文章，是同一个论证被四把不同的尺子量。
你要做的不是"写四遍"，而是**先锁死一套事实，再按四种视角各讲一遍**。

绝大多数队伍死在三件事上，全都可以提前消灭：

死法	后果	本方案的对策
四份材料数字打架	评委交叉一读，直接判"材料不实"	**单一事实源 + 一致性矩阵**（第 3、8 节）
四份材料内容重复	评委觉得注水、篇幅凑数	**分工矩阵**，一份只答一个问题（第 4 节）
佐证撑不起正文	正文越漂亮，越像编的	**三级证据设计**，L2 全覆盖（第 7 节）

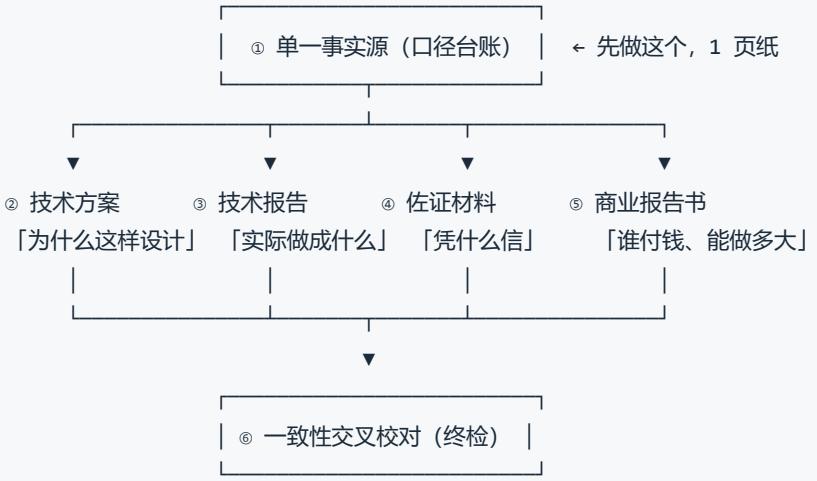
1. 评委怎么读这四份材料（四把尺子）

先想清楚"谁在读、想验证什么"，再决定"写什么"。

材料	评委的潜台词	他在找什么	一票否决点
技术方案	"你想清楚了吗？"	问题定义是否真、方案是否自洽、有无可行性论证	需求是编的、方案与问题不匹配、没有技术路线图
技术报告	"你真做出来了吗？"	实现深度、关键算法、**可复现的验证**、诚实的局限	只有截图没有数据、指标无来源、无局限性说明
佐证材料	"我凭什么信你？"	可核验、可复现、有第三方痕迹	全是自制截图、无可运行路径、时间线对不上
商业报告书	"这值钱吗？谁付钱？"	痛点强度、付费意愿、单位经济、壁垒	市场规模靠"万亿"堆、财务全靠拍、无竞品分析

关键认知：技术方案与商业报告书是**"应然"**（应该这样设计 / 应该这样赚钱）；技术报告与佐证是**"实然"**（实际做成什么样 / 凭什么证明）。应然可以宏大，实然必须精确。**越界吹牛 90% 发生在把"应然"写进了"实然"。**

2. 总架构：一源 • 四层 • 三线



三条贯穿全案的主线（四份材料必须讲同一个故事，一句不改）：

1. **主线一 • 定位**：不是"会聊天的运维助手"，是**敢让它动手的可信运维智能体。**
2. **主线二 • 机制**：**三道锁 + 一本账** —— 证据锁（回答有出处）、风险锁（高危必审批）、边界锁（能力有白名单）+ 全链路审计。
3. **主线三 • 主张**：**AI 负责思考，人负责授权，系统负责留痕。**

> 这三句要出现在：技术方案摘要、技术报告结论、佐证材料说明页、商业报告书价值主张。**同一句话，四种语气。**

3. 单一事实源：口径台账（先填这张表，再动笔）

这是整个方案的**地基**。凡是要写进任何一份材料的数字，只能从这张表里抄，不准现场想。

3.1 已核实（可直接用）

编号	事实	数值	来源	可用于
F01	Python 代码规模	**16,536 行 / 244 文件**	`agent/ knowledge/ ai-service/ backend/`	方案·报告
F02	前端代码规模	**2,245 行 / 49 文件**	`frontend/src/**/*.ts(x)`	方案·报告
F03	自动化测试	**108 个用例 / 8 个测试域**	`tests/`（api·agent·e2e·inspector·integration·knowledge·training·unit）	报告·佐证
F04	领域数据集	**7 类场景 / 14 条样本**	`SmartOps-Dataset/Linux/*.jsonl`	报告（须限定）
F05	数据状态	**全部 DRAFT（草稿，未经专家复核）**	同上 `"status":"DRAFT"`	报告（须声明）
F06	技术文档	14 页 / 8 章	`technical-document.pdf`	佐证
F07	架构角色	6 类： planner / executor / inspector / policies / kernel / tools	`agent/`	方案·报告
F08	纠察层级	**四级**： Rule → Evidence → AI → StateVerifier	`agent/inspector/pipeline.py`	报告（核心创新）
F09	竞赛 AI 技术分项	35%（LLM 8 + Agent 8 + RAG 6 + 多模态 5 + 安全 4 + MCP 4）	`README.md` L171-176	全部（对齐权重）

3.2 待测（必须补，否则技术报告没有"实然"）

编号	待测项	怎么测（低成本可行）	产出物
M01	RAG 检索质量	14 条样本 × 每样本 5 个变体问法 = 70 query；人工判定 Top-3 命中	`recall@3`、`MRR`
M02	假成功检出率	从 108 测试里构造 20 个"声称成功实则失败"场景 + 20 个真成功	混淆矩阵、P/R/F1
M03	端到端时延	20 次请求，记录首字节与完成时间	P50 / P95
M04	风险分级准确率	30 条指令人工标 4 档风险 vs 引擎输出	准确率 + 混淆矩阵
M05	人工盲评	3 位同学盲评 30 条回答（有用性/准确性/可信度 5 分制）	均分 + **Cohen's κ**

M05 是本方案最值钱的一条。 样本少不是死穴，"样本少还硬报准确率"才是。用**案例研究（case study, n=14）+ 三评者盲评 + κ 一致性**，在学术与工程评审里都站得住，成本只要一天。

3.3 禁用口径（写进任何文档都算事故）

禁写	为什么	改写成
"在 XX 数据集上准确率 9X%"	无实验记录，一问就穿	"在 14 条自建样本上完成可行性验证，Recall@3 = __（n=14，见附录 A）"
"构建了千级/万级数据集"	实际 14 条	"7 类场景的种子集（14 条），扩样方案见 §x"
"基于大模型的 Planner"	代码是规则基线	"确定性 Planner 基线 + LLM Planner 可插拔接口"
"向量数据库检索"	内存 numpy 线性扫描 + hash 嵌入	"轻量向量检索（内存索引），可平滑替换 FAISS/pgvector"
"多智能体协同"	实际是 pipeline	"多角色 Agent 流水线（Planner→Executor→Inspector）"
"已服务 X 家企业"	无合同无授权	"完成 X 次场景化验证 / 取得 X 份意向沟通"

4. 分工矩阵：一份只答一个问题

防重复的唯一办法：给每份材料划死"只能写什么"。同一素材可以在四份里出现，但**视角必须不同。**

素材	技术方案里怎么写	技术报告里怎么写	佐证里怎么呈现	商业报告里怎么写
三道锁机制	设计动机 + 机制图 + 为什么这样分层	每个锁的实现 + 拦截率数据	审批流截图 + 审计日志原文	"合规可追溯"如何变成付费理由
RAG 混合检索	为什么 0.6/0.4 加权、为什么版本过滤	公式 + 参数 + 检索指标	检索 trace 日志 + 脚本	知识资产如何形成壁垒
四级纠察	为什么需要四级、各级职责	状态机 + 假成功检出率	一次 REJECTED 的完整记录	降低"AI 事故赔付"风险
数据集	覆盖哪些运维场景	规模、构造方法、标注规范	原始 jsonl + schema	领域数据 = 护城河
技术栈	选型理由（对比表）	部署拓扑 + 资源占用	部署截图 + 启动日志	交付成本低 → 毛利高

一句话记忆：

- 方案答"**为什么**"，报告答"**是什么 + 有多好**"，佐证答"**真的吗**"，商业答"**值多少**"。

5. 逐份作战方案

5.1 技术方案（建议 18–22 页）

定位：说服评委"这个问题真、这个解法对、这个人想清楚了"。**不写实现细节，不写代码。**

目录大纲

章	内容	页	必须嵌的证据锚点
摘要	300 字讲完：问题→方案→创新→效果	1	主线三句话
1 问题定义	运维"三高"痛点：高频重复、高危误操作、高责任风险；现有 AI 助手"能说不能做、能做不敢信"	3	1 个真实事故场景（脱敏）
2 需求分析	功能需求 / 非功能需求（可信、可审计、可回滚）/ 约束	3	需求-能力映射表
3 总体方案	分层架构图 + 数据流 + 三道锁一本账	4	架构图（唯一权威版）
4 关键算法设计	① 版本感知混合检索 ② trust_score 聚合 ③ 四级纠察与假成功判定 ④ 风险分级双闸	6	4 个公式 + 4 张流程图
5 技术路线与可行性	选型对比表、风险与备选方案、里程碑	3	选型对比表
6 创新点	3 条，每条"痛点→做法→差异"	2	与竞品的差异矩阵
7 实施计划	甘特图（已完成/进行中/规划）	1	与报告的时间线一致

致命扣分点

- ❌ 架构图画得漂亮但和代码对不上 → 架构图**必须**与 `agent/` 目录结构一一对应，图下标注模块路径。
- ❌ 创新点写成"我们用了 RAG、用了 Agent" → 用 RAG 不是创新，**"版本感知 + 证据分级"才是。**
- ❌ 没有"不做什么" → 加一节"边界声明"，明确当前不支持的能力（反而加分）。

5.2 技术报告（建议 30–40 页，最厚一份）

定位：证明"我真的做出来了，而且我知道它好在哪、差在哪"。**这是四份里唯一允许出现数字的地方，也正因如此最危险。**

目录大纲

章	内容	页	硬要求
1 概述	目标、范围、与方案的对应关系	2	声明"本文只报告已实现内容"
2 系统实现	模块清单（表）+ 关键类/函数 + 数据模型	6	每个模块标注 `路径:行号`
3 关键算法实现	混合检索（0.6/0.4 + 各自归一化）、`trust_score = 0.5×均分 + 0.3×支持度 + 0.2×min(n/3,1)`、四级纠察状态机、风险双闸	8	公式 + 伪代码 + 复杂度
4 工程实现	并发、SSE 流式、沙箱、权限、审计落库	5	时序图
5 验证与实验	**本章决定成败**：实验协议 → 数据集 → 结果 → 分析	8	全部数字标 `n=` 与置信说明
6 测试	108 用例分布、覆盖模块、典型用例	3	测试矩阵表
7 部署与运行	拓扑、资源占用、启动方式	3	一键启动命令
8 局限与改进	**诚实清单**：中文未分词、规则基线、无重试幂等、AI Judge 未绑模型	3	每条给"影响 + 升级路径"
附录	复现脚本、原始数据、评测明细	4	可执行

第 5 章怎么写才不塌（核心）

采用**"三层证据结构"**，从强到弱排列，先亮最强的：

1. **可复现的功能验证**（最强）：108 个自动化用例 + 一键脚本 → 证明系统按设计工作。
2. **小样本量化评估**（次强）：M01–M04，**每个指标必须带** `n=`，并给出置信区间或"样本量不足以支撑统计显著性"的说明。
3. **人工盲评**（补充）：M05，3 评者 + Cohen's κ，κ>0.6 即可称"一致性良好"。

> **写法示范**（照抄这个句式）：
> "在 14 条自建运维样本（全部为 DRAFT 状态，未经领域专家复核）上，检索模块 Top-3 命中率为 __%（n=70 query）。受样本量限制，该结果仅用于验证链路有效性，不作为泛化性能结论。"

这句话的价值：**它同时展示了结果、暴露了局限、划清了边界**。评委看到这种表述，会认为你懂科研规范。

致命扣分点

- ❌ 有"实验结果"章节但没有原始数据/脚本 → 必须附 `评测明细.csv`。
- ❌ 只报最好的一组参数 → 给参数敏感性（至少 2 组对比）。
- ❌ 局限写成"未来会更好" → 局限要写**技术原因**（如"BM25 未分词，因零依赖约束；影响：中文长 query 召回下降，量化见 §5.4"）。

5.3 佐证材料（建议 15–25 页 + 附件包）

定位：不解释、不抒情，只提供**可核验的痕迹**。评委翻它的时间不超过 3 分钟，所以结构必须"3 秒见结论"。

三级证据体系（务必 L2 全覆盖）

级别	定义	本项目可交付物	状态
L1 自证	证明"存在"	代码仓库、目录树、模块清单、数据库表结构	✅ 现成
L2 可复现	证明"能跑"	① 一键启动脚本 ② Docker/依赖清单 ③ 3 分钟演示录屏 ④ 测试执行日志 ⑤ 评测脚本 + 原始输出	⚠️ 需补 ③④⑤
L3 他证	证明"别人也认"	① 软著/专利受理 ② 部署上线地址 ③ 用户/运维人员访谈记录 ④ 开源 star/issue ⑤ 第三方测试或 CI 记录 ⑥ 导师/企业推荐意见	❌ 全缺，**至少补 2 条**

> **L3 是分水岭。** 只有 L1+L2 的队伍是"学生作业"，有 L3 的队伍是"可信项目"。最容易补的两条：**软著申请受理号**（1–2 周可下）、**1 位一线运维人员的书面评价**（一页纸，签字即可）。

佐证册结构（按此顺序装订）

序	内容	页	要点
0	**证据索引表**	1	正文每个论断 → 对应证据编号 → 页码（**最重要的一页**）
1	系统实物	4	界面截图（带时间戳）+ 一次完整任务的端到端流程
2	代码与结构	3	目录树、模块清单、关键算法代码片段（带行号）
3	测试证据	3	108 用例清单摘要 + 执行日志截图 + 通过率
4	数据证据	2	数据集 schema、样例、标注规范、DRAFT 状态说明
5	评测证据	4	脚本 + 原始输出 + 指标计算过程（可手算复核）
6	安全与审计	3	一次高风险操作被拦截 → 审批 → 执行 → 审计日志全链路
7	部署证据	2	拓扑、启动日志、资源占用
8	第三方证明（L3）	3	软著/访谈/推荐/CI

致命扣分点

- ❌ 截图无时间戳、无上下文 → 每张图配一行"这是在验证什么"。
- ❌ 佐证里出现正文没提的东西 → 佐证只服务正文，不新增主张。

- ❌ 提交"代码压缩包"当佐证 → 附**仓库地址 + 提交哈希**，比压缩包可信 10 倍。

5.4 商业报告书（建议 20–25 页）

定位：回答"谁付钱、为什么现在、为什么是你"。**技术出身的队伍最容易在这里翻车**——把商业报告写成产品说明书。

目录大纲

章	内容	页	关键要求
摘要	一句话商业模式 + 三个数字	1	客户数 / 客单价 / 毛利
1 市场与痛点	自上而下 + **自下而上双向校验**	4	自下而上必须能算
2 产品与方案	能力地图（不讲算法，讲"帮客户省了什么"）	3	一句一个收益
3 商业模式	谁付钱、怎么收、收多少	4	订阅 / 私有化 / 按次
4 竞争分析	vs 通用大模型 / 低代码平台 / 传统 AIOps	3	**差异矩阵**，敢写"我们不如谁"
5 壁垒	4 条：约束工程 + 领域资产 + 插件生态 + 评测框架	3	明确"壁垒不是模型"
6 GTM	获客路径、首批 10 个客户从哪来	2	具体到渠道名
7 财务预测	3 年，收入/成本/毛利，**全部标注假设**	3	附敏感性分析
8 团队与风险	分工 + 3 个主要风险及对策	2	风险要真

市场规模怎么算才不被笑（重点）

❌ 错误写法："中国 IT 运维市场规模超 3000 亿，我们占 1% 就是 30 亿。"

✅ 正确写法（自下而上，每一层可查）：

```
TAM = 中国规模以上企业数 × 年均 IT 运维软件支出
SAM = 其中有专职运维团队、且存在合规审计要求的企业数 × 可承受客单价
SOM = 3 年内可触达客户数（按获客能力测算）× 客单价
```

> 三个数都要标"数据来源 / 假设依据"。**宁可 SOM 只有 3000 万，也不要 TAM 写 3000 亿。** 评委会认为前者是做过功课，后者是套模板。

财务预测的护栏

- 每个数字后面加 `（假设：__）`；
- 必须有一张**敏感性分析**：客单价 ±30%、获客成本 ×2 时，盈亏平衡点怎么变；

- 第一年不要写盈利。学生项目第一年盈利 = 明显拍脑袋。

- ❌ 竞品分析只列"我们更好" → 必须有一栏"我们不擅长"。
- ❌ 壁垒写成"技术领先" → 技术会被追上，**要写"追上来要付出的代价"**（数据积累、合规适配、客户切换成本）。
- ❌ 通篇没有"谁付钱" → 每一页都要能回答"这句话跟收钱有什么关系"。

---

6. 三条红线与标准话术（背下来）

这三条是本项目**特有的**风险，评委只要真读材料就一定会碰。

红线一：数据集只有 14 条，且全是 DRAFT

危险表述：任何形如"准确率 9X%"的句子。

标准话术（书面）：

> "本项目构建了覆盖 7 类 Linux 运维场景的种子数据集（14 条），当前状态为 DRAFT，用于打通'数据—检索—纠察—执行'全链路。本文报告的检索指标基于该种子集的小样本评估（n=70 query），**仅用于验证链路有效性，不构成泛化性能结论**。扩样与专家复核方案见 §8.2。"

标准话术（口头）：

> "样本量小是我们最清楚的短板。所以我们没有报准确率，而是报'链路是否打通'和'失败案例在哪'——这两件事 14 条样本足够说明。"

红线二：实验标记为 NOT RUN

危险表述：报告里出现空白的"实验结果"表格，或用"预计""理论上"填数。

标准话术：

> "我们的评测分为已完成与未完成两部分，报告中只呈现已完成部分，未完成部分明确标注 NOT RUN 并给出执行协议。**知道哪些结论还不能下，比什么都敢写更可信。**"

红线三：多个组件是"规则基线"

涉及：Planner（规则）、BM25（未分词）、向量检索（内存 + hash 嵌入）、AI Judge（可插拔未绑模型）。

统一表述模板：

"该模块当前为**确定性基线实现**，设计目标是先保证可测、可复现、零外部依赖；接口已按可替换设计（`__init__` 注入 / 策略模式），生产环境可平滑切换为 LLM Planner / jieba 分词 / FAISS 索引，替换成本见 §8.3。"

为什么这样答反而加分：评委见惯了"我们用了最先进的大模型"，见到"我们知道哪里是基线、以及为什么先做基线"，会认为你**工程判断力成熟**。

---

7. 佐证材料清单（照着打勾）

L2 必补（本周内可完成）

- ☐ `scripts/run-all.sh` 一键启动（后端 + AI 服务 + 前端）
- ☐ `scripts/eval_retrieval.py` 评测脚本 + 输出 `eval_results.csv`
- ☐ `scripts/eval_false_success.py` 假成功混淆矩阵脚本
- ☐ 3 分钟演示录屏（含一次高危拦截 + 审批 + 审计回看）
- ☐ 测试执行日志（`pytest -q` 完整输出，存为 txt）
- ☐ `requirements.lock` / `package-lock.json` 依赖锁定
- ☐ 数据库表结构导出（`schema.sql`）
- ☐ 系统架构图源文件（drawio/svg，证明不是截图拼的）

L3 必补（至少 2 条）

- ☐ 软件著作权申请受理通知书（**最快、最硬**）
- ☐ 1 位一线运维工程师的书面评价（1 页，含签名/工牌打码）
- ☐ 部署上线地址（哪怕是临时公网 IP + 只读账号）
- ☐ 指导教师推荐意见（1 页）
- ☐ GitHub/Gitee 仓库公开 + commit 历史（证明开发过程真实）
- ☐ CI 运行记录截图

---

8. 一致性矩阵：同一事实的四种写法

这张表是防"交叉打脸"的最终武器。 定稿后逐行核对。

事实	技术方案（应然）	技术报告（实然）	佐证（客观）	商业报告（价值）
系统能力	"设计支持 6 类运维任务闭环"	"已实现 __ 类，__ 类规划中"	界面截图 + 任务清单	"可替代 __% 重复工单"
检索效果	"预期显著降低版本错配"	"Top-3 命中 __%（n=70）"	检索 trace 日志	"知识复用率提升"
安全机制	"三道锁设计"	"四级纠察 + 双闸，拦截 __/__"	拦截记录原文	"降低事故赔付风险"
数据资产	"7 类场景覆盖"	"14 条种子，DRAFT"	原始 jsonl	"领域数据资产（早期）"
性能	"目标 P95 < 3s"	"实测 P95 = __s（n=20）"	时延日志	"单实例可支撑 __ 并发"
团队	"6 人分工"	作者贡献声明	commit 记录	团队履历

核对口诀：**方案说"要"，报告说"了"，佐证说"证"，商业说"值"。** 只要有一份越界（方案写了"了"，或商业写了"证"），就是漏洞。

9. 七天作战排期（含依赖关系）

写作顺序 ≠ 提交顺序。**必须先有事实源，否则四份一起返工。**

天	任务	产出	依赖
D1 上午	填口径台账（第 3 节）+ 补 M01–M05 评测	`fact-sheet.md` + `eval_results.csv`	无
D1 下午	冻结架构图、模块清单、创新点三条	架构图 v1 + 创新点卡	台账
D2	**技术方案**定稿（18–22 页）	`01-技术方案.docx`	D1
D3–D4	**技术报告**主体（2–4 章实现 + 5 章验证）	`02-技术报告.docx` 初稿	方案（骨架复用）
D5	**佐证材料**装订 + 补 L2/L3	`03-佐证材料.pdf`	报告（摘事实）
D6	**商业报告书**（市场/模式/财务）	`04-商业报告书.docx`	报告（摘能力）
D7 上午	**一致性交叉校对**（第 8 节逐行）+ 格式统一	四份终稿	全部
D7 下午	打印装订 + 备份 U 盘 + 答辩稿同步更新	提交包	终稿

格式统一清单（D7 必做，最容易被扣"不规范"分）

- ☐ 四份封面同一模板：项目名、队名、学校、日期、版本号
- ☐ 页码连续，页眉统一
- ☐ 图表全部编号（图 1-1、表 2-3）并在正文引用

- [ ] 术语表统一（"纠察"不写成"检查"/"校验"混用）
- [ ] 参考文献格式统一（GB/T 7714）
- [ ] 中英文标点统一、无错别字（用工具过一遍）

10. 交付前 30 项自检

内容层

- [ ] 1. 四份材料的"一句话主张"完全一致
- [ ] 2. 所有数字都能在口径台账里查到
- [ ] 3. 所有指标都带 `n=`
- [ ] 4. 没有任何"准确率 9X%"式无源结论
- [ ] 5. 数据集状态明确写 DRAFT
- [ ] 6. 规则基线组件全部标注"基线 + 可插拔"
- [ ] 7. 技术报告有独立的"局限"章节，且写技术原因
- [ ] 8. 商业报告有"我们不擅长"一栏
- [ ] 9. 财务数字全部标注假设 + 敏感性分析
- [ ] 10. 市场规模为自下而上测算，且来源可查

结构层

- [ ] 11. 四份目录与本文档大纲一致（或差异有理由）
- [ ] 12. 技术方案的架构图 = 技术报告的模块图 = 代码目录
- [ ] 13. 佐证索引表覆盖正文全部关键论断
- [ ] 14. 每份都有摘要，且摘要能独立成立
- [ ] 15. 无跨文档复制粘贴的长段落

证据层

- [ ] 16. L2 证据 8 项全齐
- [ ] 17. L3 证据 ≥ 2 项
- [ ] 18. 所有截图带时间戳或上下文说明
- [ ] 19. 提供仓库地址 + commit 哈希
- [ ] 20. 评测脚本可被第三人跑通（找同学实测一次）

格式层

- [ ] 21. 封面信息完整（队名/学校/队员/指导教师/日期）

- [ ] 22. 页码、页眉、目录自动生成
- [ ] 23. 图表编号连续且被引用
- [ ] 24. 参考文献格式统一
- [ ] 25. 无错别字、无占位符 `【待填】` 残留

答辩衔接

- [ ] 26. 答辩稿里的每个数字都能在四份材料中找到出处
- [ ] 27. 答辩稿的"创新点"与技术方案第 6 章逐字对应
- [ ] 28. 答辩稿的"局限"与技术报告第 8 章逐字对应
- [ ] 29. 演示脚本覆盖佐证材料里的关键截图
- [ ] 30. 打印一份纸质版，**自己从头翻一遍**（一定能发现电脑上看不出的问题）

---

11. 我需要你确认的 4 件事

这四点会直接改变四份材料的写法，越早定越省返工：

1. **提交截止时间**是哪天？——决定第 9 节排期是压到 3 天还是放宽到 10 天。
2. **格式与页数限制**（秩序册里通常有"不超过 X 页/A4"）——PDF 是扫描件我读不出文字，你截图那一页给我即可。
3. **是否匿名评审**——匿名则全文不能出现学校名、导师名、仓库真实地址，佐证也要脱敏。
4. **团队真实分工**——技术报告要写作者贡献，商业报告要写团队履历，两处必须一致，且最好与 commit 记录对得上。

附：本文档与既有材料的关系

已有文件	用途	与本方案的关系
`docs/competition/technical-document.pdf`	14 页技术文档	**技术报告的骨架**，扩写为 30–40 页
`docs/competition/defense.md`	陈述稿	从四份材料的"摘要"提炼，保持同源
`docs/competition/defense-master.md`	70 题问答	答辩追问的弹药库，**内容须与四份材料一致**
`docs/competition/defense-advanced.md`	12 张底牌 + 技术追问	红线话术来源（第 6 节）
`docs/competition/答辩材料-打印合订本.pdf`	答辩打印册	提交后更新，加入四份材料的核心数字

执行原则：先产出口径台账 → 再写技术方案 → 其余三份从方案与报告中"派生"，而不是各自从零写。**派生比分头写快 3 倍，且天然一致。**